Practical Task Document – Podcast Script Generation System (AI/ML + Python)

1) Purpose

Build a system that generates a **conversational podcast script** between **two speakers (Host and Guest)**. The script must be generated from **user-uploaded documents**, **selected topics**, and a **target duration**.

Candidates may use **any LLM** (Open Source API call from HuggingFace).

2) Required Implementation (Streamlit UI + Working Backend)

2.1 Interface Requirement

Can use **Streamlit UI** to collect inputs and run the flow end-to-end.

If the Streamlit UI is incomplete, the **core code must still run successfully via terminal/CLI/ swagger** and produce the final script output.

> ***Final output correctness matters more than UI polish.***

3) Functional Flow

3.1 Pre-Flow Setup (Inputs Before "Start")

Collect the following inputs via Streamlit UI or Swagger API:

A) Speaker Information (Mandatory)

- **Host Name** (text)
- **Guest Name** (text)
- **Host Gender** (male/female)
- **Guest Gender** (male/female)
- **Host Speaking Speed** (50–150 scale, slow → fast)
- **Guest Speaking Speed** (50–150 scale, slow → fast)

B) Document Upload (Mandatory)

User must upload **at least one** document from:

- **PDF**
- **DOC/DOCX**
- **TXT**

These documents are the source for topic extraction and script grounding.

C) Approximate Duration (Mandatory)

Select one:

- 5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60 minutes

Validation

If any mandatory field is missing, block the flow and show a **single consolidated error message**.

3.2 Topic Selection

1. System extracts **candidate topics** from all uploaded documents.
2. UI displays extracted topics for the user to review.
3. User selects topics to include.
4. If the user enters extra topics manually:
    - Show:
        - **Topics included** (found in documents)
        - **Topics ignored** (not found in documents)

5. User can:
    - Approve and continue
    - Restart flow (reset all inputs and selections)

3.3 Script Generation (Core)

Generate a full **Host–Guest dialogue script** based on:

- Selected topics
- Document content
- Speaker genders + speaking speeds (use for realism and pacing)
- Target duration (approximate)

Script Requirements

- Natural, humanoid conversation style:
    - light fillers like “um”, “uh”, “hmm” in moderation
    - realistic back-and-forth
    - smooth transitions
- Must include:
    - opening
    - topic coverage with transitions
    - closing

Review + Modification Loop

- Script is shown to the user for review.
- User may request modifications using prompts.

- Each regeneration must output a **complete script** (no partial edits).
- Unlimited modification cycles allowed.

4) Edge Cases & Handling

1. **No documents uploaded** → block with error.
2. **Document too thin / no extractable topics** → notify user and request additional document(s); restart flow.
3. **Topic mismatch** → always show “Included vs Ignored”.
4. **Restart flow** → reset all prior inputs; user must re-enter configuration.

5) Evaluation Criteria

We will evaluate:

1. **Consistency & coherence**
    - logical dialogue, no contradictions, stable tone/personality
2. **Topic coverage**
    - selected topics are covered meaningfully and grounded in documents
3. **Humanoid conversational quality**
    - sounds like a real host/guest conversation
4. **Length approximation**
    - script length reasonably matches selected duration
5. **System behavior**
    - validations, edge cases, restart, and modification loop work correctly

6) Deliverables

1. Python project with:
    - Streamlit UI (Optional)
    - Backend logic for parsing docs, topic extraction, script generation
2. README with:
    - setup steps
    - how to run UI (If streamlit used)
    - how to run via terminal (fallback)
    - LLM used and required environment variables/keys (no secrets committed)
3. Sample output script(s) for quick review